

Attack Monitoring and Blocking: From Transaction Frontrunning to Secure Sequencer

By Yajin Zhou

Professor of Zhejiang University, CEO of BlockSec

<https://yajin.org>

<https://blocksec.com>

The results presented in these slides are a team effort and could not have been achieved without the contributions of my students at Zhejiang University and colleagues at BlockSec.

About Me

- Co-founder of [BlockSec](#) and Professor of Zhejiang University
- Research interests
 - DeFi security, Blockchain system security
- Publish: 60+ papers with 9,000+ citations
- Hack and build systems
- Read more: <https://yajin.org>

Security Matters in Web3



Despite the bull and bear cycles in the crypto market, losses caused by exploits and scams have been **growing at a rapid pace.**

Security Matters in Web3

Explorer > Security Incidents

Security Incidents

The attack incidents causing losses exceeding \$100K will be documented.

[All](#) [Bookmark](#)

<https://app.blocksec.com/explorer/security-incidents>

Search For Project Name or Tx Hash of Attack Tx

	Project	Loss	Chain	Vulnerability	Date		
+	Ronin Bridge ☆	~ \$11.8 M		Misconfiguration	2024/08/06	 Root Cause	< Share
+	TokenStake (Unknown) ☆	~ \$578 K		Vulnerable Price Dependency	2024/08/05	 Root Cause	< Share
+	Convergence ☆	~ \$210 K		Unverified User Input	2024/08/01	 Root Cause	< Share
+	Spectra ☆	~ \$550 K		Arbitrary Call	2024/07/23	 Root Cause	< Share
+	DeltaPrime ☆	~ \$1 M		Misconfiguration	2024/07/22	 Root Cause	< Share
+	UPS ☆	~ \$521 K		Business Logic Flaw	2024/07/21	 Root Cause	< Share
+	WazirX ☆	~ \$207 M		Compromised Private Key	2024/07/18	 Root Cause	< Share
+	LI.FI ☆	~ \$11 M		Arbitrary Call	2024/07/16	 Root Cause	< Share
+	Minterest Finance ☆	~ \$1.5 M		Reentrancy	2024/07/14	 Root Cause	< Share

Why Security Incidents are Prevalent

- Economical incentive
 - Hackers can get “paid”. Think about a house full of gold but without a good security system
- Less security-qualified developers
 - Developers are not trained well in security concepts
- DeFi composability: creates more attack vectors

Why Security Incidents are Prevalent

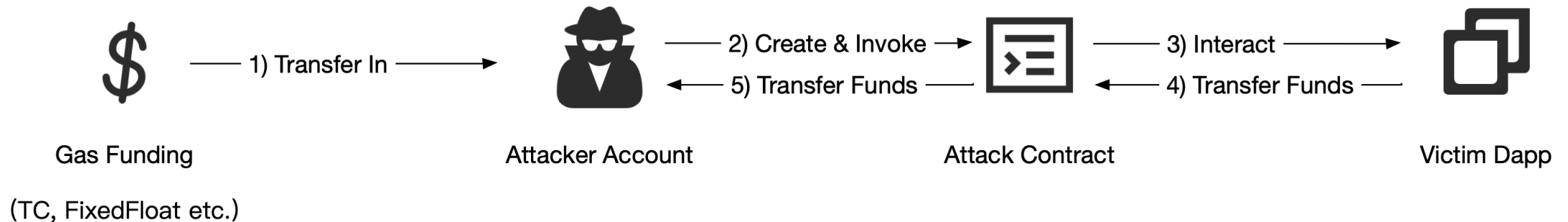
- Openness: everyone can see the code on the chain, and everyone can issue an attack tx if a vulnerability exists
- Anonymity: it's hard to trace (not impossible) the attacker if he/she is smart enough to hide
- Flashloan: enlarge the money that an attacker can use

Existing Approach

- Pre-launch
 - Code auditing
 - Fuzzing testing
 - Formal verification
- Post-launch
 - We think there should be an effective approach to detect and **block** hacks after the protocol is deployed

Steps of a DeFi Attack

- Prepare the attack
- **Launch the attack**
- Launder the profit

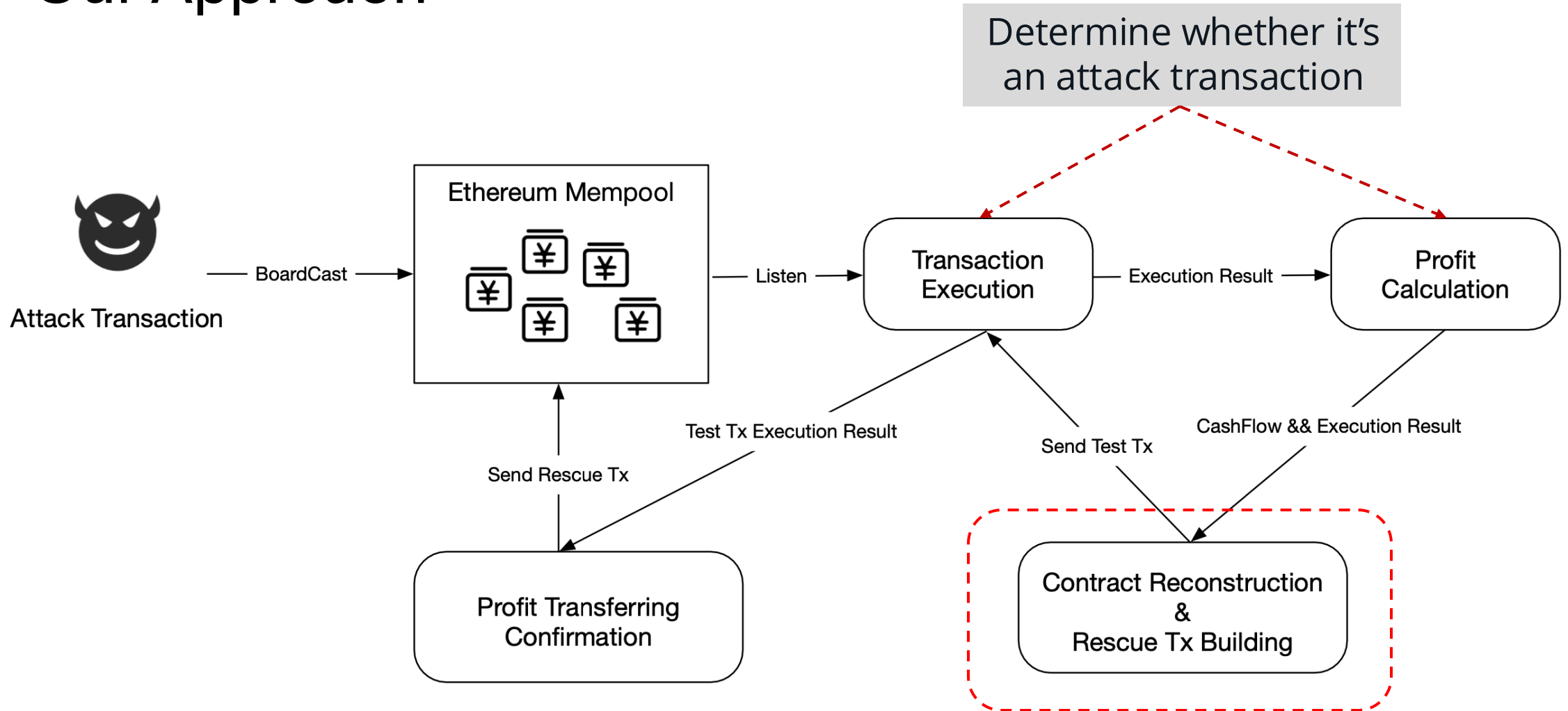


Our Approach

- The process of a transaction is confirmed
 - T1: A transaction is broadcasted to the p2p network
 - T2: Every node on the network can listen to pending transactions
 - T3: Validators confirm the transaction

Can we **detect and block** the attack transaction during the time window of T1 and T3 (typically 12 seconds in Ethereum)

Our Approach



Challenges and Solutions

- How to extract attack logic from the attack contract
 - The attack contract contains verification logic (of the caller) – in a simple way or in an obfuscated way
 - Callback functions

Challenges and Solutions

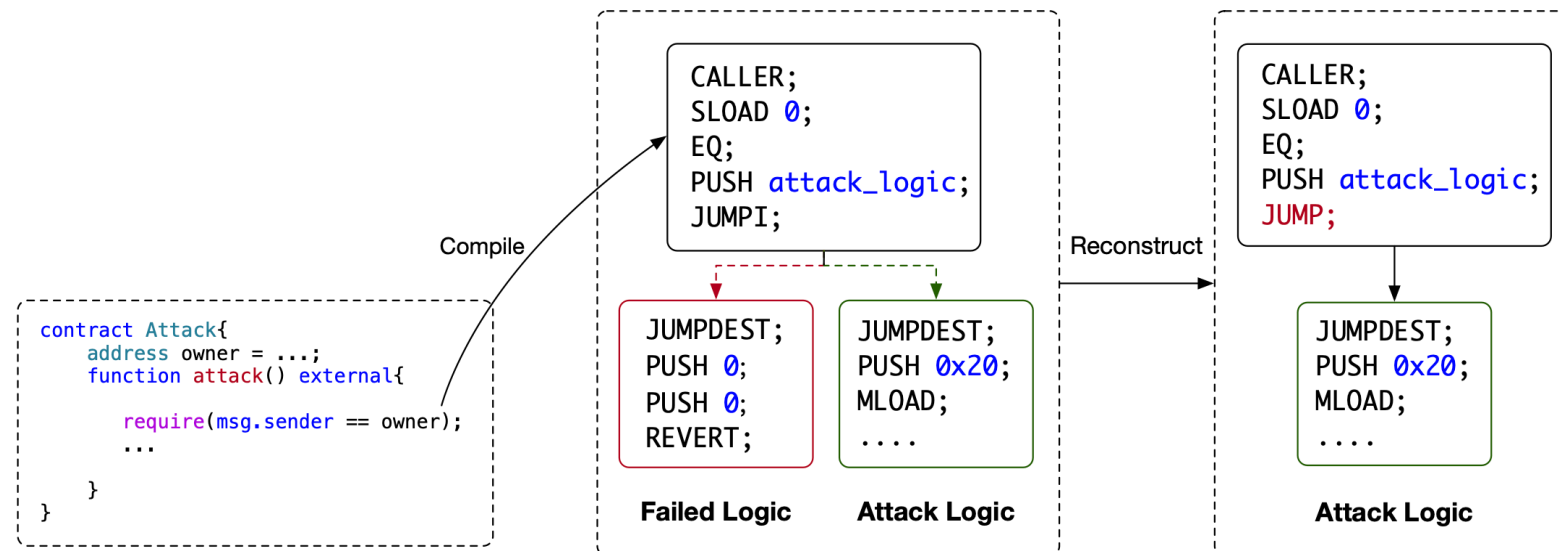
- How to locate and replace revenue addresses
 - Revenue addresses are the ones that get profit during the attack
 - We need to replace them with our ones

Challenges and Solutions

- How to identify the pre-conditions of an attack
 - Auxiliary contracts
 - Multiple transactions to launch an attack

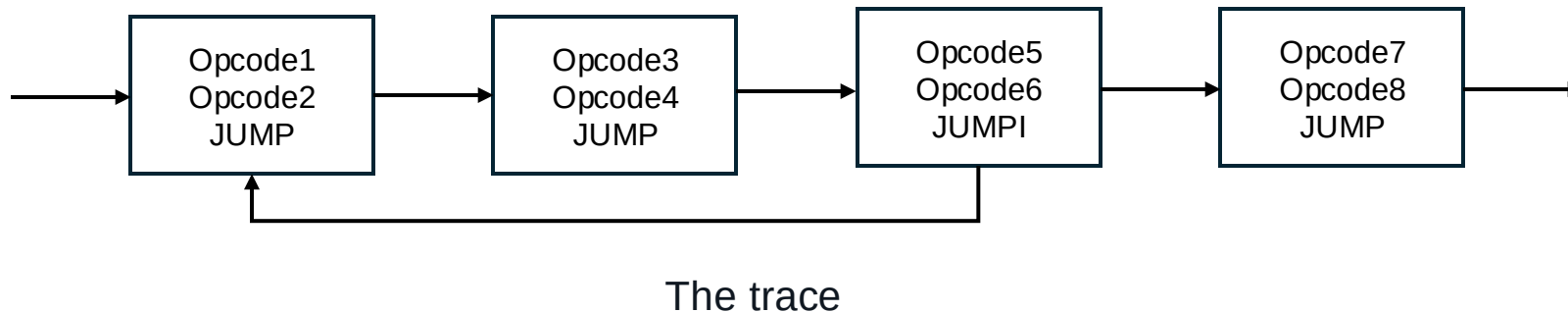
Task One: Extract and Construct Attack Logic

- Automatically reconstruct a smart contract with the attack logic
 - Attacks usually use some logic to protect their attack contacts



Task One: Extract and Construct Attack Logic

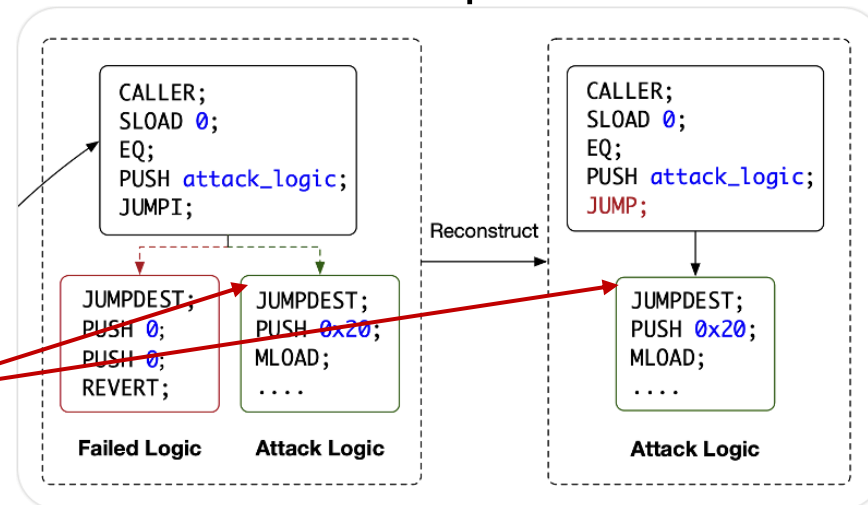
- Option one: connect the opcode trace of an attack transaction
 - Input: opcode trace of an attack transaction
 - Output: a new smart contract with the trace



Generated contract

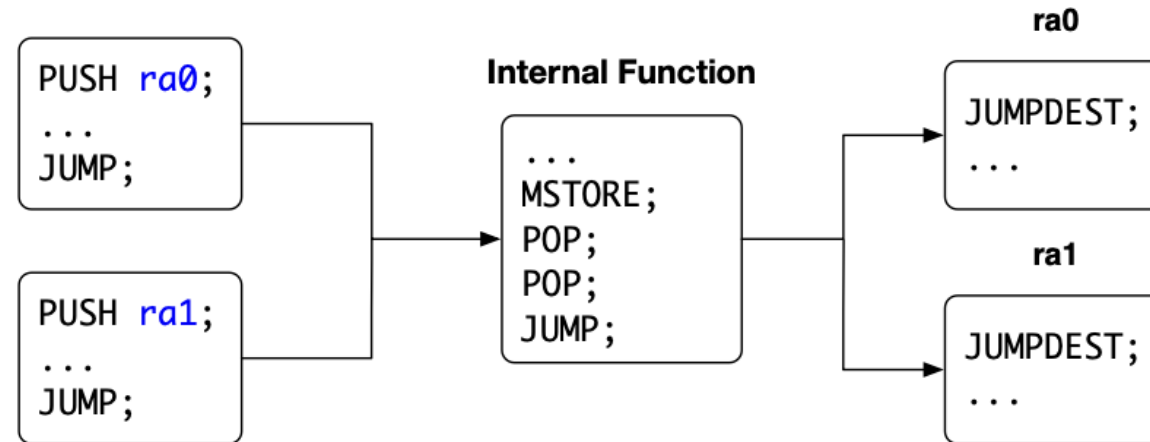
```
Opcode1
Opcode2
Opcode3
Opcode4
Opcode5
Opcode6
Opcode1
Opcode2
Opcode3
Opcode4
Opcode5
Opcode6
Opcode7
Opcode8
```


- Option two: reuse the basic block as much as possible
 - It works theoretically and in practice
 - But the process is more complicated than the previous one
 - We need to handle the JUMP instruction and fixup the offsets in the trace and our generated contract



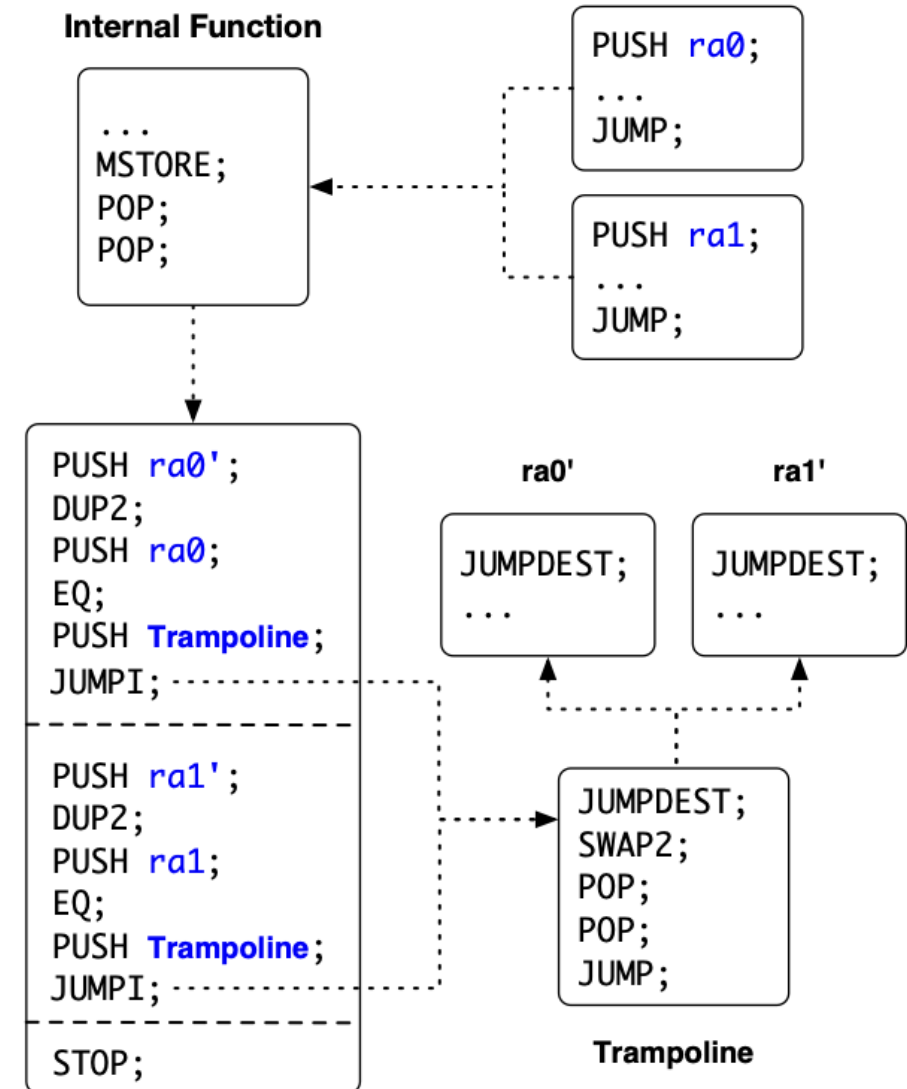
Freezing Conditional JUMP

- For JUMP with multiple targets
 - The real target is determined in the runtime -> from which basic block



Freezing Conditional JUMP

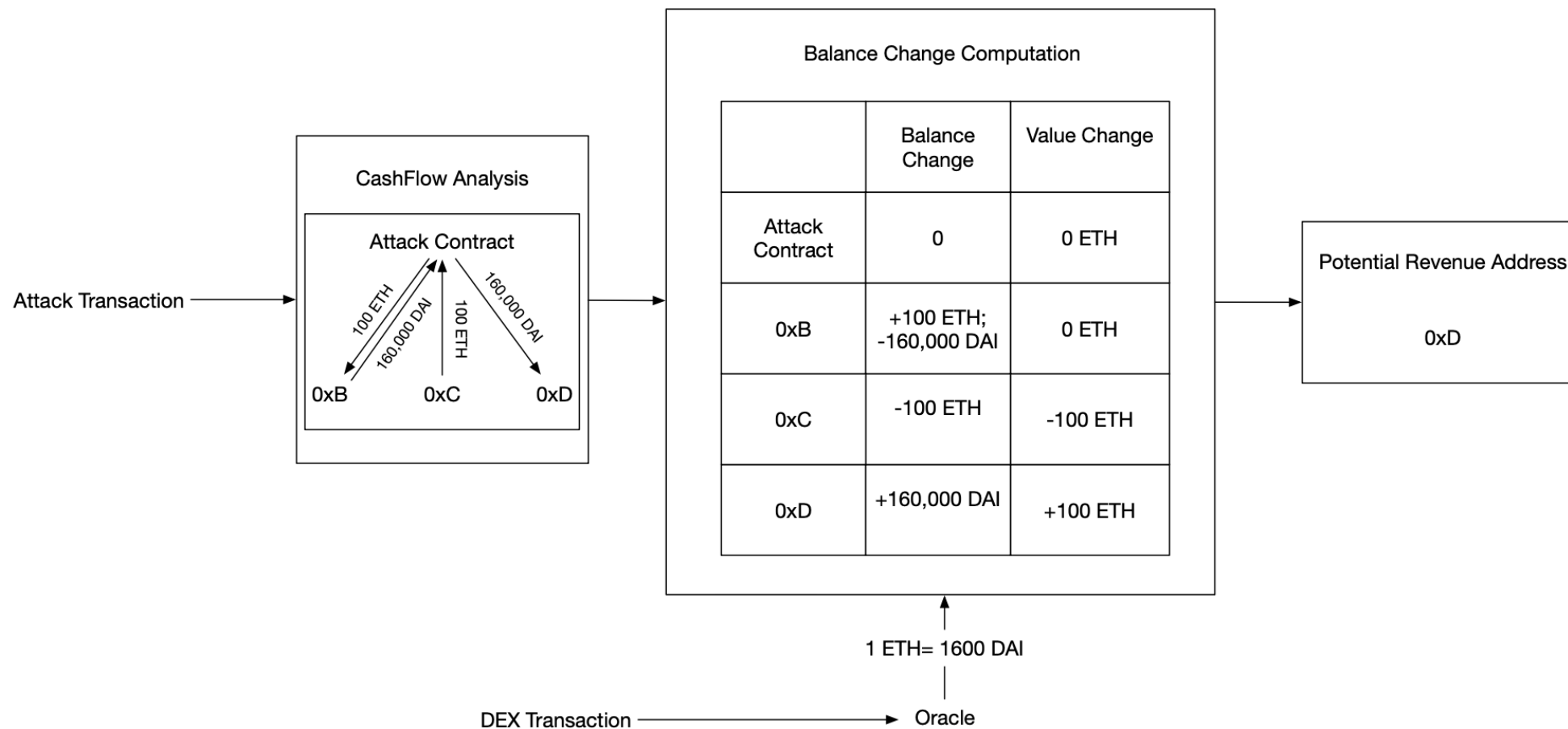
- We compare the basic block and use a trampoline to relocate the destination block
 - $ra0 \rightarrow ra0'$
 - $ra1 \rightarrow ra1'$
- Also, we need to make the stack balance after the trampoline



Task Two: Identify and Replace Revenue Addresses

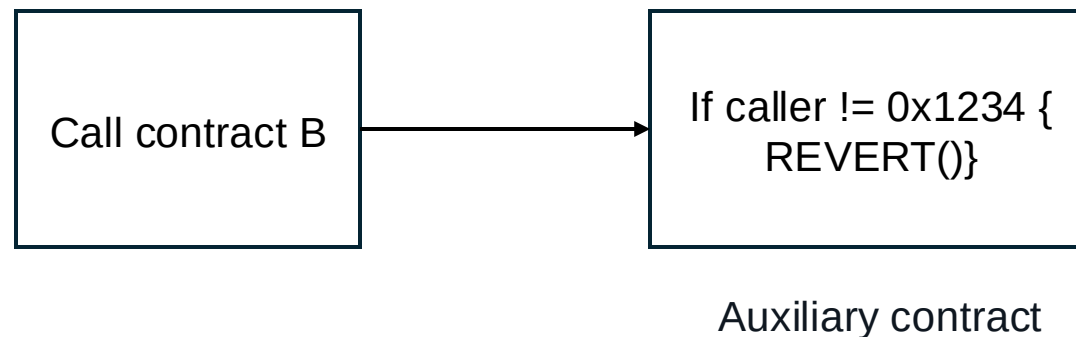
- Revenue addresses
 - The ones that balance changes positively, *and replacing them does not affect the attack logic*
- How to identify them: try-and-catch
 - Calculate the balance changes
 - Replace the ones with positive balance change to see what happens
- When: Replace them when they are pushed on the stack

Task Two: Identify and Replace Revenue Addresses



Identify Auxiliary contracts

- Created during the attack: find CREATE/CREATE2 instruction
- Created before the attack: use REVERT to locate them
 - The auxiliary will revert during the interaction (since we change the caller when invoking the auxiliary contract)
 - We track all the reverted internal calls to identify the auxiliary contract



- Attackers can split the attack transaction into multiple ones



Identify Multiple Transactions

- Our method
 - We retrieve the state dependency search method to locate all preparation transactions
 - State-dependent: a transaction is using a storage changed by a previous transaction
 - Optimizations: cache state change called by a tx in (24 hours)

Evaluation

- Effectiveness
 - If our system can successfully synthesize the attack tx and contract with replaced revenue addresses, then our system is effective in blocking that hack
- Dataset
 - Historical attacks in [DeFiHackLabs](#): June 2020 to February 2023
 - Real attacks

Evaluation

- Dataset: 117 attacks in total, we use 87 of them
 - The attack crosses different blocks
 - They require some special tokens
- Among 87 attacks, we can successfully generate the attack contracts

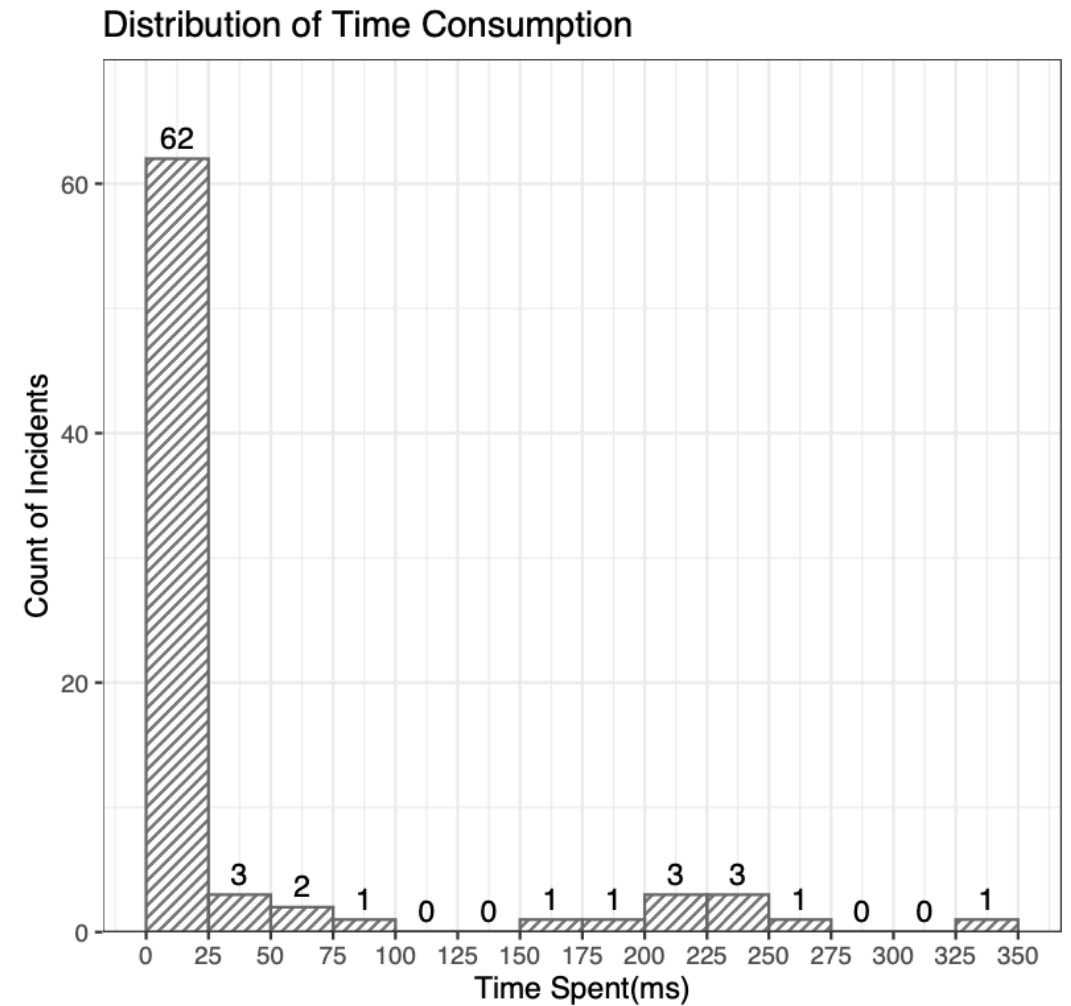
Feasibility	Category	Success	Total
Not Feasible	Unable to finish in a single block	0	29
	Require special capital	0	4
Feasible	Common attack	47	50
	Involve anti-front-running strategy	31	34

Evaluation

- Reasons for failed cases
 - Attack tx was too complicated with many auxiliary contracts – the generated contract was too big (bigger than the maximum allowed contract size)

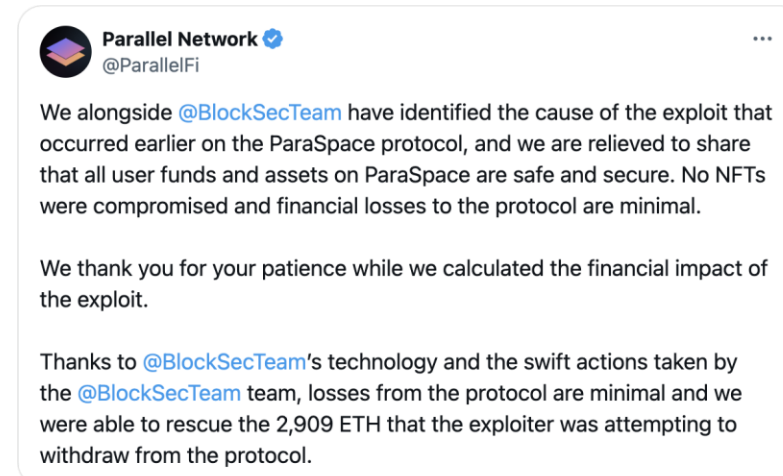
Evaluation

- Efficiency
 - How quick to reconstruct the attack contract
 - More than 80% cases are finished in less than 25 ms



Real Cases

- Blocked hacks and rescued more than 20 million USDs
 - Representative ones: ParaSpace: 5 Million, Saddle Finance: 3.8 million



<https://blocksec.com/blog/lead-in-phalcon-s-hack-blocking-saga>

From Frontrunning to Phalcon

- The technique is good, but ...
 - What about the legal issue
 - What about the competition between our system and other MEV bots
- We need another solution from the perspective of a product

BlockSec Phalcon

- A platform to monitor and **block** hacks
 - Hack blocking tx can be **triggered automatically** from the protocol
 - The key to the system is to **monitor the hacks with low false positives and low false negatives**

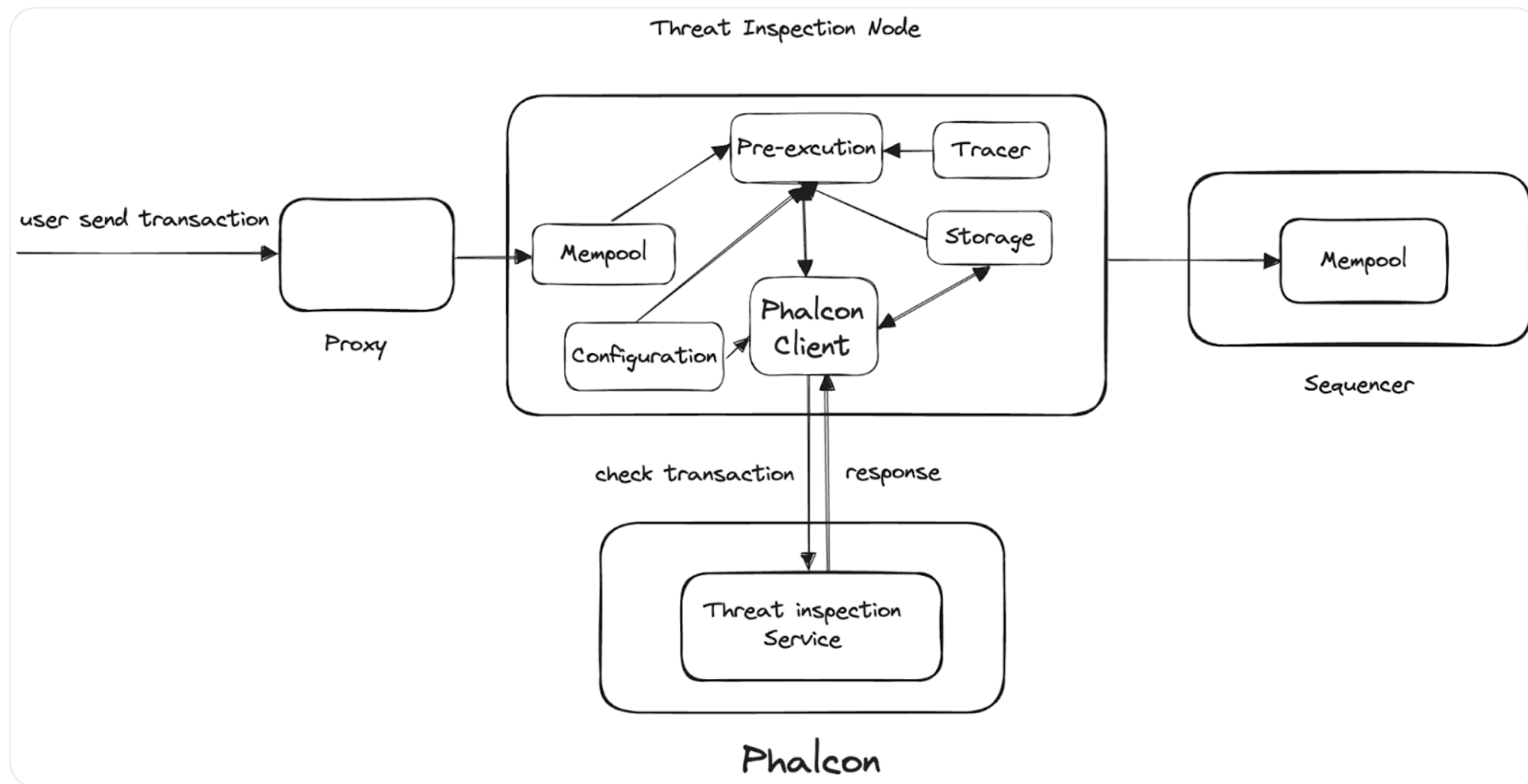
BlockSec Phalcon

- How to reduce false positives
- How to reduce false negatives

From Phalcon to STOP (**S**equencer **T**hreat **O**verwatch **P**rogram)

- STOP is the solution to secure L2 chains
- Sequencer plays the role of validator
 - Responsible for packing the transactions inside a block
- We can enable attack detection and blocking inside sequencer

STOP (Sequencer Threat Overwatch Program)



From Phalcon to STOP (**S**equencer **T**hreat **O**verwatch **P**rogram)

- Challenges
 - Attack detection accuracy
 - Latency
 - Transactions from L1 -> L2

From Phalcon to STOP (**S**equencer **T**hreat **O**verwatch **P**rogram)

- The first L2 with inborn sequencer-level security



About BlockSec

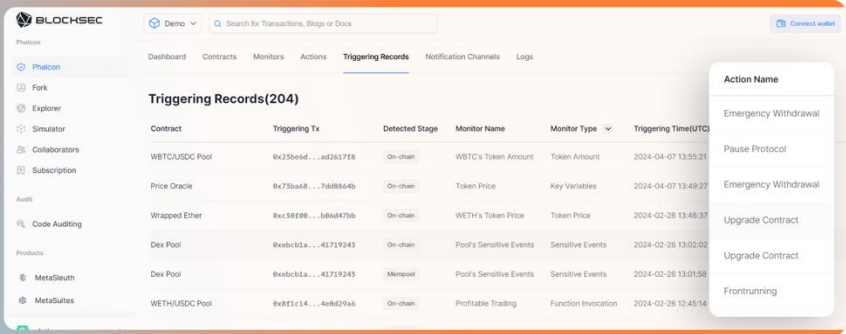
- Full stack security providers
- For protocols and chains
 - Pre-launch: code auditing
 - Post-launch
 - Attack monitoring and blocking: Phalcon
 - Crypto tracking and investigation: MetaSleuth (<https://metasleuth.io>)

BlockSec Phalcon

- BlockSec Phalcon (<https://blocksec.com/phalcon>)
 - Gain early access to precise attack intelligence
 - Automated attack blocking with customized actions

Automated Actions

Implement swift and automatic responses to threats.



The screenshot displays the BlockSec Phalcon interface. On the left is a sidebar with navigation options: Phalcon, Fork, Explorer, Simulator, Collaborators, Subscription, Audit, Code Auditing, Products, MetaSleuth, and MetaSalties. The main area shows the 'Triggering Records(204)' table with columns: Contract, Triggering Tx, Detected Stage, Monitor Name, Monitor Type, and Triggering Time(UTC). A dropdown menu for 'Action Name' is open, listing: Emergency Withdrawal, Pause Protocol, Emergency Withdrawal, Upgrade Contract, Upgrade Contract, and Frontrunning.

Contract	Triggering Tx	Detected Stage	Monitor Name	Monitor Type	Triggering Time(UTC)
WSTC/USDC Pool	0x250e6d...ad1617f8	On-chain	WSTC's Token Amount	Token Amount	2024-04-07 13:55:21
Price Oracle	0x750a68...7d48864b	On-chain	Token Price	Key Variables	2024-04-07 13:49:27
Wrapped Ether	0xc38f00...b0d472b	On-chain	WETH's Token Price	Token Price	2024-02-26 13:48:37
Dex Pool	0xebc31a...41719245	On-chain	Pool's Sensitive Events	Sensitive Events	2024-02-26 13:02:02
Dex Pool	0xebc31a...41719245	Mempool	Pool's Sensitive Events	Sensitive Events	2024-02-26 13:01:58
WETH/USDC Pool	0x8f1c14...4e8d29a6	On-chain	Profitable Trading	Function Invocation	2024-02-26 12:43:14

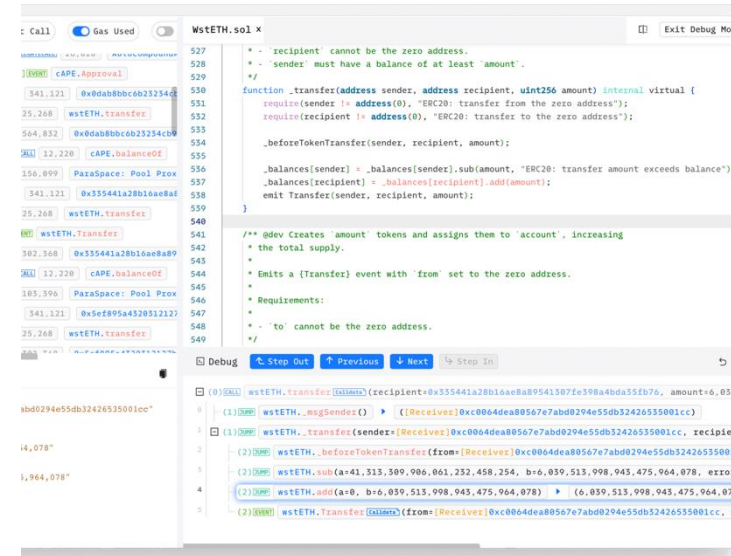
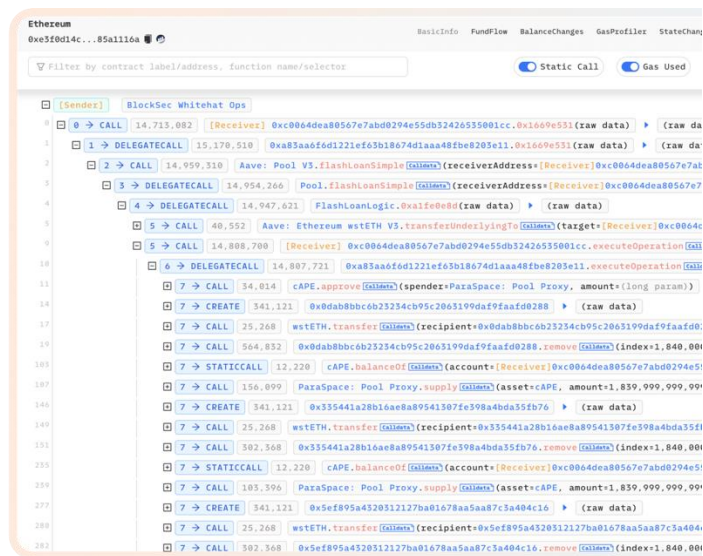
MetaSleuth

- <https://metasleuth.io>



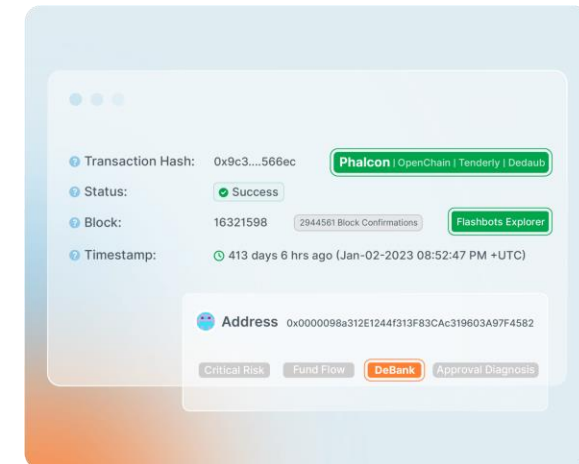
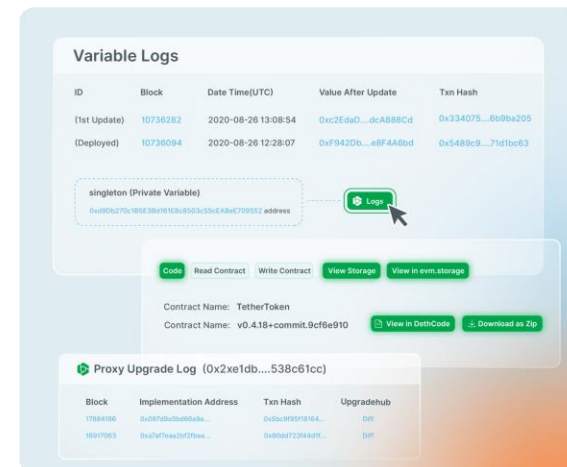
Phalcon Explorer

- Free transaction analysis tool for security/MEV researchers – Solana is supported - <https://blocksec.com/explorer>



MetaSuites

- A free extension for builders (<https://blocksec.com/metasuities>)
 - Address labels
 - Fund flows
 - Aggregator for popular web3 tools
 - Development accelerator
 - It supports **Solana**



<https://blocksec.com>
<https://metasleuth.io>

Twitter:
[@BlockSecTeam](https://twitter.com/BlockSecTeam)
[@phalcon_xyz](https://twitter.com/phalcon_xyz)
[@MetaSleuth](https://twitter.com/MetaSleuth)

contact@blocksec.com

YouTube Channel



Twitter



<https://www.youtube.com/@blocksecteam>